

Resumen de aprendizaje

Construccion de agente_2.py (AgenteFDA)

1. Acceder a una clave de diccionario (datos["results"]) vs .values()

Fallo:

Pensabas que `.values()` te daría "todo el contenido útil" del diccionario que devuelve la API.

Solucion:

`datos["results"]` accede directamente a la clave que sabes que contiene la lista de recalls.

Por que:

`.values()` devuelve TODOS los valores del diccionario mezclados (incluido el bloque meta, que no querías), no solo el que te interesa.

2. .info() no devuelve datos, solo imprime

Fallo:

Intentabas guardar `df1.info()` en una variable para exportarlo luego.

Solucion:

Construir tu propio resumen con `df1.dtypes`, `df1.count()` y `df1.isnull().sum()`, que si devuelven datos reales.

Por que:

`.info()` esta pensado para leerlo en consola mientras programas, no para procesarlo despues: su valor de retorno es `None`.

3. Un groupby() sin agregacion final no es una tabla

Fallo:

Usabas `df1.groupby("classification")["reason_for_recall"]` esperando que ya fuera un resultado.

Solucion:

Cerrarlo con `.count()`, `.sum()`, etc.

Por que:

`groupby()` solo agrupa: es un objeto "en espera" hasta que le dices que hacer con cada grupo.

4. pd.DataFrame(a, b, c, d) no combina varios resultados

Fallo:

Pasabas cuatro resultados distintos como argumentos seguidos, esperando que se fusionaran en una tabla.

Solucion:

Usar un diccionario (`{"clave": resultado, ...}`) en vez de forzar un unico DataFrame.

Por que:

Los primeros parametros posicionales de `pd.DataFrame()` son `data`, `index`, `columns`, `dtype`: no "junta estas cosas", cada argumento tiene un papel fijo distinto.

5. Exportar varios resultados a un mismo Excel: diccionario + pd.ExcelWriter + .items()

Fallo:

No sabias como meter varias tablas distintas en un solo archivo .xlsx.

Solucion:

with pd.ExcelWriter(ruta) as writer: y un bucle for nombre_hoja, tabla in diccionario.items(): tabla.to_excel(writer, sheet_name=nombre_hoja).

Por que:

Cada .to_excel() suelto sobrescribe el archivo entero; el writer mantiene el archivo "abierto" mientras escribes una hoja por cada entrada del diccionario.

6. Como fluyen los datos entre funciones (return + parametros)

Fallo:

No entendias de donde "salia" la variable resumen que recibia exportar_resultados.

Solucion:

return en una funcion solo entrega el valor a quien la llamo; hay que capturarlo en una variable y pasarlo explicitamente a la siguiente funcion.

Por que:

El nombre del parametro dentro de una funcion es solo una etiqueta local: no esta relacionado por nombre con la variable de fuera, solo por la posicion en la llamada.

7. Atributo vs metodo: .dtypes sin parentesis

Fallo:

Escribiste df1.dtypes(), como si fuera una funcion.

Solucion:

df1.dtypes sin parentesis.

Por que:

dtypes es un atributo (un dato que ya existe), no un metodo que haya que "ejecutar": el mismo tipo de error que ya habias cometido antes con .shape() en numpy.

8. Elegir la agregacion correcta segun el tipo de dato

Fallo:

Usaste .sum() sobre una columna de texto (classification).

Solucion:

.count(), para contar cuantos recalls hay por grupo.

Por que:

.sum() sobre texto concatena strings en vez de sumar numeros: no da un error, pero tampoco el resultado que buscabas.

9. Indentacion: el cuerpo de un bloque va mas adentro que su cabecera

Fallo:

Pegaste el cuerpo de interpretar_analisis (y luego el if __name__) al mismo nivel que su def/cabecera.

Solucion:

Cuatro espacios mas de sangria para el cuerpo de una funcion; cero espacios para el bloque de entrada, porque va fuera de la clase.

Por que:

En Python la indentacion no es estetica, es la sintaxis que delimita que pertenece a que: un desajuste rompe la carga de todo el archivo (IndentationError), no solo esa linea.

10. if __name__ == "__main__": no es parte de la clase

Fallo:

Pensabas que este bloque era "la conversacion" o que debia ir dentro de AgenteFDA.

Solucion:

Es el punto de entrada del script: crea el objeto y llama a sus metodos en orden, mientras que la conversacion real vive dentro de interpretar_analisis.

Por que:

Una clase define comportamiento (que sabe hacer un agente); algo tiene que decidir cuando y en que orden usarlo, y eso vive fuera, a nivel de modulo.

11. Dependencias y librerias deprecadas

Fallo:

El script asumia que dotenv, google.generativeai y openpyxl ya estaban instaladas.

Solucion:

pip install python-dotenv google-generativeai openpyxl.

Por que:

Importar una libreria que no esta instalada da ModuleNotFoundError antes de ejecutar nada mas; de paso descubrimos que google.generativeai esta descatalogado a favor de google.genai.

12. El directorio de trabajo (cwd) afecta a rutas relativas y a load_dotenv()

Fallo:

Dabas por hecho que "darle a Play" en VS Code ejecutaria el script desde su propia carpeta.

Solucion:

Ejecutar desde terminal entrando primero a la carpeta (cd Agentes/agente_2) para tener control real de desde donde corre.

Por que:

Path(" analisis_es") y load_dotenv() son relativos al directorio desde el que se lanza el script, no a donde esta guardado el archivo: si no coinciden, no encuentra el .env o crea carpetas en el sitio equivocado.
